Project Executable Files

Date	29 September 2026
Team ID	SWTID-2026-3161
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
College	Balagan Saraswathi Arts and Science College For Women (MSU144)
Team Leader	Muthu Iswarya C
Team Members	a Sowmya S, Dharshini M, Esakiammal A

Project Executable Files

This section lists all executable and deployable files for the project "Script-Controlled ACL – Restrict Record Access Based on Field Value". The solution is built and configured on the ServiceNow platform, so the evaluator can reproduce it using the steps below.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes – ACL script and ServiceNow configuration (Read, Create, Write, Delete ACLs)
2	README / Setup Guide (instructions to run the project)	Yes – see Step 4 below and the project report
3	requirements.txt / package.json / dependency file	NA – configured directly in ServiceNow, no external dependencies
4	Database schema / seed files (if applicable)	Yes – u_institution_details table with fields and sample records (ECE, EEE, CSE)
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	Yes – ServiceNow instance URL (see Step 3)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA

9	Test files and test results	Yes – impersonation test screenshots (bb1, bb1+bb2, bb1+bb2+bb3, all roles, admin)
10	Demo video or walkthrough (if required)	Yes – link in Step 3 (add before submitting)

Step 2: File / Folder Structure

Overview of the project's file and folder structure:

```
/Script-Controlled-ACL-Project
/1_Users_and_Roles      - EEE User details, roles bb1, bb2, bb3, bb4
/2_Table_Schema         - u_institution_details table and field list
/3_ACL_Configuration    - Read, Create, Write, Delete ACL details
   read_acl_script.js   - Script used in the Read ACL
/4_Test_Results         - Impersonation screenshots and verification
/5_Documentation        - NM_document_fff.pdf (project report)
README.md               - Setup and run instructions
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://<your-instance>.service-now.com (replace with your ServiceNow instance URL)
Login Credentials (Demo)	Admin: <username> / <password> EEE test user: EEE User / <password>
Platform / Hosting Provider	ServiceNow (Personal Developer Instance)
Repository Link	<GitHub / Drive link to project files>
Demo Video Link	<link to demo video>

Step 4: Run Instructions

Step-by-step instructions for the evaluator to run the project:

Pre-requisites:

ServiceNow instance (Personal Developer Instance) with admin access, a modern web browser (Chrome / Edge), and the security_admin role available for elevation.

Step 1: Log in to the ServiceNow instance with admin access. User Administration → Users → New. Create user: User ID “EEE User”, First name “EEE”, Last name “User”, Email eeeuser@gmail.com. Save.

Step 2: Create the custom roles. User Administration → Roles → New. Create bb1, bb2, bb3 and bb4, then assign all four roles to EEE User.

Step 3: Create the custom table. System Definition → Tables → New. Label: Institution Details, Name: u_institution_details. Add fields: Student Roll Number (Auto Number), Student Name (Reference–User), Faculty Name (Reference–User), Branch (Choice: ECE, EEE, CSE), Email, Phone Number (String), Description (Multi-line String).

Step 4: Add sample records. Create multiple records with different Branch values (ECE, EEE, CSE).

Step 5: Elevate role. Click the profile icon → Elevate role → select security_admin → Update.

Step 6: Create the Read ACL. System Security → Access Control (ACL) → New. Type: record, Operation: read, Name: u_institution_details, Active: true, Advanced: true. Requires role: bb1. Data condition: Branch is EEE. Paste the script (below) in the Script field and Submit.

Step 7: Create the Create ACL. New ACL: Operation: create, Name: u_institution_details, Requires role: bb2, no data condition. Submit.

Step 8: Create the Write ACL. New ACL: Operation: write, Name: u_institution_details, Requires role: bb3, no data condition. Submit.

Step 9: Create the Delete ACL. New ACL: Operation: delete, Name: u_institution_details, Requires role: bb4, no data condition. Submit.

Step 10: Test using impersonation. Impersonate EEE User, open u_institution_details.list and verify access. Then impersonate Admin and verify all records are visible.

Script used in the Read ACL:

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }
  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')) {
    return true;
  }
  // Deny access for all others
  return false;
})();
```

Expected results:

• bb1 only: sees only EEE branch records (no New button).

• bb1 + bb2: sees EEE records and the New button.

• bb1 + bb2 + bb3: can also edit EEE records.

• bb1 + bb2 + bb3 + bb4: can also delete EEE records.

• User without the roles: “No records to display”. Admin: sees all records.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	The script checks only the role (bb1); the EEE restriction comes from the Data condition (Branch is EEE) set on the ACL.	Keep the Data condition “Branch is EEE” active on the Read ACL.
2	Creating or editing ACLs requires the security_admin role to be elevated.	Elevate the role from the profile menu before creating ACLs.
3	A user needs the roles in sequence (bb1 for viewing, bb2 create, bb3 edit, bb4 delete); without bb1 no records are visible.	Assign all required roles (bb1–bb4) to the test user.
4	Branch or role changes require updating the ACL configuration manually.	Update the ACL data condition / roles when requirements change.